Does size matter?

Investigating the Impact of Weight Regain on Fight Outcomes in the UFC

In [66]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import openpyxl
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LogisticRegression


from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix

import math

import statsmodels.formula.api as smf

from sklearn.metrics import roc_curve, roc_auc_score
from scipy.stats import chi2
from statsmodels.stats.diagnostic import linear_harvey_collier
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.proportion import proportions_chisquare
from statsmodels.stats.api import het_breuschpagan

import statsmodels.api as sm
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.stats.outliers_influence import summary_table
from statsmodels.stats.diagnostic import linear_reset

from statsmodels.stats.diagnostic import lilliefors
from statsmodels.stats.diagnostic import het_white

from sklearn.metrics import roc_auc_score, roc_curve
from statsmodels.stats.diagnostic import het_breuschpagan

from statsmodels.stats.proportion import proportions_chisquare
from statsmodels.stats.api import het_white

from statsmodels.stats.diagnostic import linear_harvey_collier


from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.api import het_white
```

```
from statsmodels.stats.diagnostic import het_white
import os
print(os.environ['PATH'])
```

```
/Users/caseymoser/opt/anaconda3/envs/UFC_data/bin:/Users/caseymoser/opt/anac
onda3/condabin:/usr/bin:/bin:/usr/sbin:/sbin
```

Background

One of the most pervasive controversies in fighting is weight cutting. Fighters are divided into weight classes that they choose to fight in for the sake of fairness. However, fighters have realized they can gain a massive advantadge by reducing their weight through water cutting — the process of temporarily losing weight through sweat and regaining it post weigh-in — to make weight classes that otherwise would be impossible for their body composition. Since weigh-ins are done the day before fights, fighters can then recover the lost water and go back to their "real" weight by fight night. The advantadge of weight cutting, especially against an opponent who cuts less, is drastic. Fighters that weigh more hit harder, can more easily smother their opponents in wrestling, and can have a signficant reach advantadge.

Because of this signficant advantadge from weight cutting, fighting has increasingly become a game of who can cut the most weight. Fighters have been known to regain up to 20 lbs between weigh in and fight night, with the highest recorded regain from Geoff Neal, who gained 30.3 pounds at UFC 298 (https://www.espn.com/mma/story/_/id/39610394/seven-ufc-298-fighters-flagged-rehydration-issue).

On one hand, this phenomenon hurts the sport by making fighting skill less important to the overall equation. Some fighters who may be less skilled are able to win fights because their bodies are naturally adapted to rapidly losing and gaining water weight, a term disparagingly reffered to as "weight bullying." Furthermore, exciting fights can get cancelled from fighters failing to make weight from attempting too large of a weight cut, or even having to pull out due to health complications from bad water cuts (https://www.mmamania.com/2022/9/9/23345292/ufc-279-dana-white-reveals-khamzat-chimaevs-weight-cut-ended-after-locking-and-cramping). Even more concerning is the danger that comes with weight cutting. Fighters have been known to not only have serious health complications from bad weight cuts, but have even died from it (https://www.espn.com/mma/story/_/id/14344041/chinese-mma-fighter-yang-jian-bing-dies-trying-make-weight).

To combat this isue, organizations such as One Championship, a rival organization to the UFC, have implemented measures such as measuring hydration levels post weigh in to ensure fighters are not cutting too much water weight. However the UFC, the biggest and most popular MMA organization in the world, currently has no such measures.

Due to how pervasive weight cutting has become, it is important to investigagte how crucial weight cutting has become to fighting. In this study, I intend to investiage the impact weight cutting has on the odds of winning fights.

Explanation of the Data and the Data Sources

The data used for fight results is up to date as of UFC 318 which took place on 7/19/25. The data set can be found here: https://github.com/Greco1899/scrape_ufc_stats

For context, the weight classes in the UFC are as follow:

- Flyweight – up to 125 lb
- Bantamweight – over 125 to 135 lb
- Featherweight – over 135 to 145 lb
- Lightweight – over 145 to 155 lb
- Welterweight – over 155 to 170 lb
- Middleweight – over 170 to 185 lb
- Light Heavyweight – over 185 to 205 lb
- Heavyweight – over 205 to 265 lb

Independent Variable

Weight regain percentage: To measure a fighter's weight cut, I have made a variable called PERCENT_REGAIN. This variable is the percentage change between a fighter's weigh-in weight and their fight weight the next day. This fight-night weight is unfortunately not universal across UFC events because not every region measures it. For example, the California State Athletic Commission (CSAC) measures fight night weight for the purposes of regulating extreme weight cuts. This data has been collected from the dataset linked here: https://www.reddit.com/r/MMA/comments/evbnjd/released_offical_ufc_fight_night_weights/. It is important to note that this data is from 6 year ago so fights since then have not been capture in it. Weight differences are measured as a percentage to better account for the fact that the higher weight classes have the room to lose more absolute weight compared to the lighter weights.

To then compare the differences in fighter weight regains, I created a follow-up variable called PERCENT_REGAIN_DIFF. This variable measures the percentage point difference

between Fighter A and their opponent Fighter B. If PERCENT_REGAIN_DIFF is positive, it means that Fighter A regained a greater percentage of their own weight compared to Fighter B. In similar fashion, a negative value for PERCENT_REGAIN_DIFF indicates that Fighter B (which is the classified as the opponent in my data set) regained a greater percetange of their own weight compared to Fighter A.

Hypothesis

My hypothesis is that greater percentage of weight regained will be associated with higher odds of winning fights by giving the fighter a height and weight advantadge on fight night.

In [2]:
```
stats_path = '/Users/caseymoser/Desktop/UFC Analysis/UFC/Weight Analysis/ufc

# data source: https://www.reddit.com/r/MMA/comments/evbnjd/released_offical
weight_path = '/Users/caseymoser/Desktop/UFC Analysis/UFC/Weight Analysis/UF

results_path = '/Users/caseymoser/Desktop/UFC Analysis/UFC/Weight Analysis/u

stats_df = pd.read_csv(stats_path)

weight_df = pd.read_excel(weight_path)

results_df = pd.read_csv(results_path)
```

In [3]:
```
import pandas as pd

# Step 1: Split fighters into separate columns
results_df[['Fighter_1', 'Fighter_2']] = results_df['BOUT'].str.split(' vs.

# Step 2: Prepare Fighter 1 dataframe
fighter1_df = results_df.copy()
fighter1_df['FIGHTER'] = fighter1_df['Fighter_1']
fighter1_df['OPPONENT'] = fighter1_df['Fighter_2']   # Add opponent
fighter1_df['RESULT'] = fighter1_df['OUTCOME'].str[0].map({'W': 'Win', 'L':
fighter1_df = fighter1_df.drop(columns=['Fighter_1','Fighter_2'])

# Step 3: Prepare Fighter 2 dataframe
fighter2_df = results_df.copy()
fighter2_df['FIGHTER'] = fighter2_df['Fighter_2']
fighter2_df['OPPONENT'] = fighter2_df['Fighter_1']   # Add opponent
fighter2_df['RESULT'] = fighter2_df['OUTCOME'].str[2].map({'W': 'Win', 'L':
fighter2_df = fighter2_df.drop(columns=['Fighter_1','Fighter_2'])

# Step 4: Combine both into a single dataframe
results_df_clean = pd.concat([fighter1_df, fighter2_df], ignore_index=True)

# Step 5: Extract UFC event number
results_df_clean['UFC_EVENT'] = results_df_clean['EVENT'].str.extract(r'(UFC
```

```
results_df_clean['FIGHT_PAIR'] = results_df_clean.apply(
    lambda x: tuple(sorted([x['FIGHTER'], x['OPPONENT']])), axis=1
)

results_df_clean['FIGHT_KEY'] = results_df_clean['UFC_EVENT'] + '_' + result


results_df_clean['FIGHT_ID'] = pd.factorize(results_df_clean['FIGHT_KEY'])[0

results_df_clean = results_df_clean.drop(columns=['FIGHT_PAIR', 'FIGHT_KEY']
```

In [4]:
```
results_df_clean['RESULT']
```

Out[4]:
```
0         Win
1         Win
2        Loss
3        Loss
4         Win
         ...
16423    Loss
16424    Loss
16425    Loss
16426    Loss
16427    Loss
Name: RESULT, Length: 16428, dtype: object
```

In [5]:
```
# Some fighters in this data set are mistakenly classified because a parathe

import re

# Function to remove trailing parentheses with numbers
def clean_fighter_name(name):
    if pd.isna(name):
        return name
    return re.sub(r'\s*\(\d+\)$', '', name).strip().lower()

# Apply to fighter and opponent columns in weight_df
weight_df['FIGHTER'] = weight_df['FIGHTER'].apply(clean_fighter_name)

# Standardize UFC_EVENT
weight_df['UFC_EVENT'] = weight_df['EVENT'].str.extract(r'(UFC \d+)', expanc

# Verify since Moicano was incorrectly labeled with a 2
weight_df[weight_df['FIGHTER']=='renato moicano']
```

Out[5]:

	EVENT	FIGHTER	WEIGHT CLASS	WEIGH IN WEIGHT (lbs)	FIGHT NIGHT WEIGHT (lbs)	WEIGHT INCREASE (lbs)	PERCENTAGE REGAINED	SE
16	1- UFC 227	renato moicano	Featherweight	146.0	165.5	19.5	13.356164	
189	8- UFC 311	renato moicano	Lightweight	155.0	181.8	26.8	17.290323	

In [6]:

```
import pandas as pd
import re

# --------------------------------
# Step 0: Clean fighter and opponent names
# --------------------------------
def clean_fighter_name(name):
    if pd.isna(name):
        return name
    return re.sub(r'\s*\(\d+\)$', '', name).strip().lower()

for df in [results_df_clean, weight_df]:
    df['FIGHTER'] = df['FIGHTER'].apply(clean_fighter_name)
    if 'OPPONENT' in df.columns:
        df['OPPONENT'] = df['OPPONENT'].apply(clean_fighter_name)
    df['UFC_EVENT'] = df['UFC_EVENT'].str.strip().str.upper()

# --------------------------------
# Step 1: Merge fighter weight info
# --------------------------------
merged_weight = pd.merge(
    results_df_clean,
    weight_df[['FIGHTER','UFC_EVENT','WEIGH IN WEIGHT (lbs)', 'FIGHT NIGHT W
    on=['FIGHTER','UFC_EVENT'],
    how='left',
    suffixes=('_result','_weight')
)

# --------------------------------
# Step 2: Assign FIGHT_ID based on fighter, opponent, event
# --------------------------------
merged_weight['FIGHT_PAIR'] = merged_weight.apply(
    lambda x: tuple(sorted([x['FIGHTER'], x['OPPONENT']])), axis=1
)
merged_weight['FIGHT_ID'] = pd.factorize(merged_weight['UFC_EVENT'] + '_' +
merged_weight = merged_weight.drop(columns=['FIGHT_PAIR'])

# --------------------------------
# Step 3: Merge opponent weights and regain (including fight night weight)
# --------------------------------
opponent_df = merged_weight[['FIGHT_ID','FIGHTER','WEIGH IN WEIGHT (lbs)','F
    columns={
        'FIGHTER':'OPPONENT',
```

```
        'WEIGH IN WEIGHT (lbs)':'OPPONENT_WEIGHT',
        'FIGHT NIGHT WEIGHT (lbs)':'OPPONENT_FIGHT_NIGHT_WEIGHT',
        'PERCENTAGE REGAINED':'OPPONENT_PERCENTAGE_REGAINED'
    }
)

merged_weight_clean = pd.merge(
    merged_weight,
    opponent_df,
    on=['FIGHT_ID','OPPONENT'],
    how='left'
)

# --------------------------------
# Step 4: Optional — drop rows with truly missing fighter weight
# --------------------------------
merged_weight_clean = merged_weight_clean.dropna(subset=['WEIGH IN WEIGHT (l

# --------------------------------
# Step 5: Compute weight and percent regain differences
# --------------------------------
merged_weight_clean['PERCENT_REGAIN_DIFF'] = merged_weight_clean['PERCENTAGE
merged_weight_clean['WEIGHT_DIFF'] = merged_weight_clean['WEIGH IN WEIGHT (l
merged_weight_clean['ABS_WEIGHT_DIFF'] = merged_weight_clean['WEIGHT_DIFF'].

# --------------------------------
# Example: filter by fighter
# --------------------------------
merged_weight_clean[merged_weight_clean['FIGHTER']=='renato moicano']
```

Out[6]:

	EVENT	BOUT	OUTCOME	WEIGHTCLASS	METHOD	ROUND	TIME
37316	UFC 311: Makhachev vs. Moicano	Islam Makhachev vs. Renato Moicano	W/L	UFC Lightweight Title Bout	Submission	1	4:05
57044	UFC 227: Dillashaw vs. Garbrandt 2	Cub Swanson vs. Renato Moicano	L/W	Featherweight Bout	Submission	1	4:15

2 rows × 25 columns

In [7]:
```
win_weight_df = merged_weight_clean

win_weight_df['RESULT'] = win_weight_df['RESULT'].map({'Win': 1, 'Loss': 0})
```

In [8]:
```
win_weight_df[win_weight_df['FIGHTER']=='renato moicano']

win_weight_df
```

```
win_weight_df[win_weight_df['FIGHTER']=='renato moicano']

win_weight_df
win_weight_df = win_weight_df.dropna(subset=['OPPONENT_WEIGHT'])
```

In [9]:
```
win_weight_df = win_weight_df.dropna(subset=['OPPONENT_WEIGHT'])
```

In [ ]:
```

```

In [ ]:
```

```

In [10]:
```
# Remove duplicate rows based on BOUT, FIGHTER, and OPPONENT
win_weight_df_clean = win_weight_df.drop_duplicates(subset=['BOUT', 'FIGHTER

# Verify
win_weight_df_clean[['BOUT','FIGHTER','OPPONENT']].duplicated().sum()  # Sho


win_weight_df_clean = win_weight_df_clean.dropna(subset=['RESULT'])


win_weight_df_clean = win_weight_df_clean.rename(columns={'PERCENTAGE REGAIN
```

In [11]:
```
import re

# Function to clean weight class using the text in WEIGHTCLASS
def clean_weight_class(row):
    wc = row['WEIGHTCLASS']
    if pd.isna(wc):
        return None

    # Determine if it's a women's division
    is_women = bool(re.search(r"women", wc, flags=re.IGNORECASE))

    # Remove extra words like "Bout", "Title", "UFC", but keep "Women" for p
    wc_clean = re.sub(r"(Bout|Title|UFC)", "", wc, flags=re.IGNORECASE).stri

    # Keep only the main weight class word
    wc_clean = wc_clean.split()[-1] if len(wc_clean.split()) > 0 else wc_cle

    # Add "Women's" prefix if it is a women's division
    if is_women:
        wc_clean = f"Women's {wc_clean}"

    return wc_clean

# Apply to the dataframe
win_weight_df_clean['WEIGHTCLASS_CLEAN'] = win_weight_df_clean.apply(clean_w

# Verify
win_weight_df_clean[['WEIGHTCLASS', 'WEIGHTCLASS_CLEAN']].head(10)
```

Out[11]:

	WEIGHTCLASS	WEIGHTCLASS_CLEAN
910	UFC Lightweight Title Bout	Lightweight
911	UFC Bantamweight Title Bout	Bantamweight
912	Light Heavyweight Bout	Heavyweight
913	Heavyweight Bout	Heavyweight
914	Middleweight Bout	Middleweight
915	Bantamweight Bout	Bantamweight
916	Middleweight Bout	Middleweight
917	Light Heavyweight Bout	Heavyweight
919	Women's Bantamweight Bout	Women's Bantamweight
920	Bantamweight Bout	Bantamweight

In [33]:
```
# Drop duplicates based on UFC_EVENT and BOUT
df_no_duplicates_clean = win_weight_df_clean.drop_duplicates(subset=["UFC_EV

df_no_duplicates_clean = df_no_duplicates_clean.reset_index(drop=True)


# New variable to show absolute difference in weight regained
df_no_duplicates_clean["ABSOLUTE_REGAIN_DIFF"] = df_no_duplicates_clean["FIG


# New variable to show difference in percentage weight regained (in percenta
df_no_duplicates_clean["PERCENT_REGAIN_DIFF"] = df_no_duplicates_clean["PERC


# Drop rows where WEIGHTCLASS_CLEAN is exactly 'Weight' (this is caused by a
df_no_duplicates_clean = df_no_duplicates_clean[df_no_duplicates_clean['WEIG
```

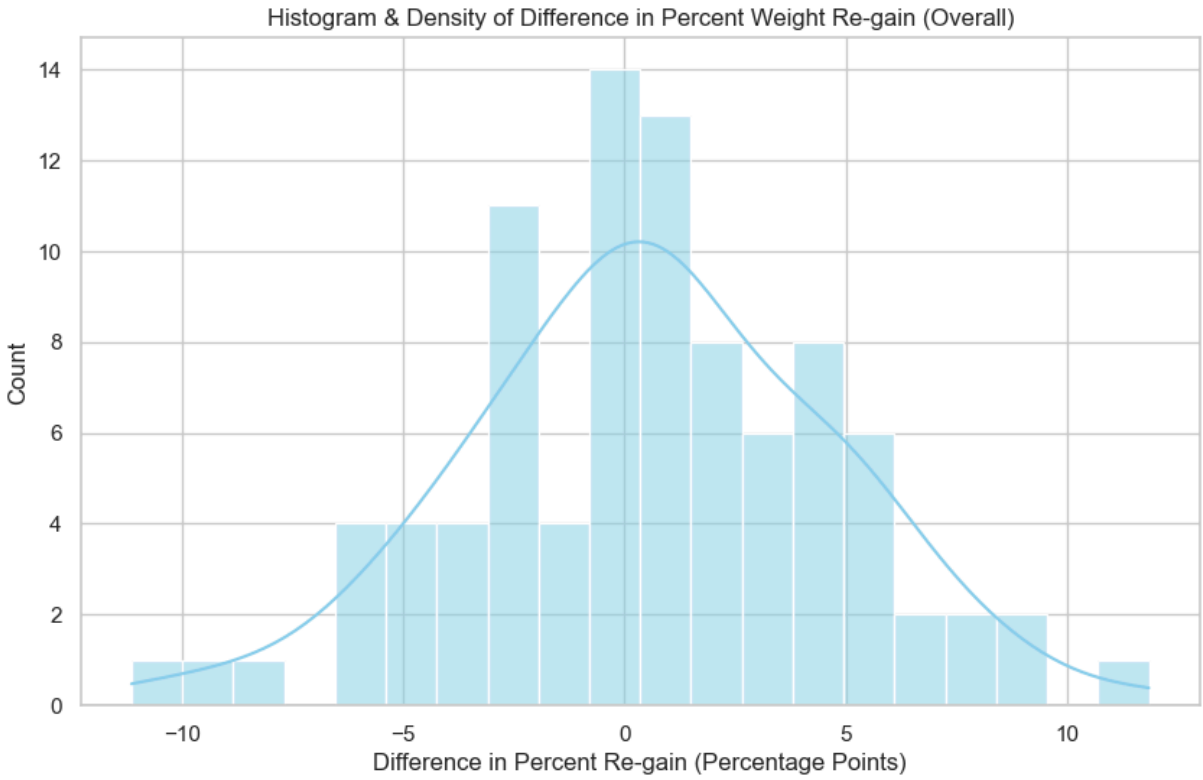
Out[33]:
```
0     15.210356
1     17.014925
2      1.809291
3     -1.191489
4      3.760218
        ...
89    13.246397
90    16.269841
91    10.701107
92     6.108787
93     9.806452
Name: PERCENT_REGAIN, Length: 92, dtype: float64
```

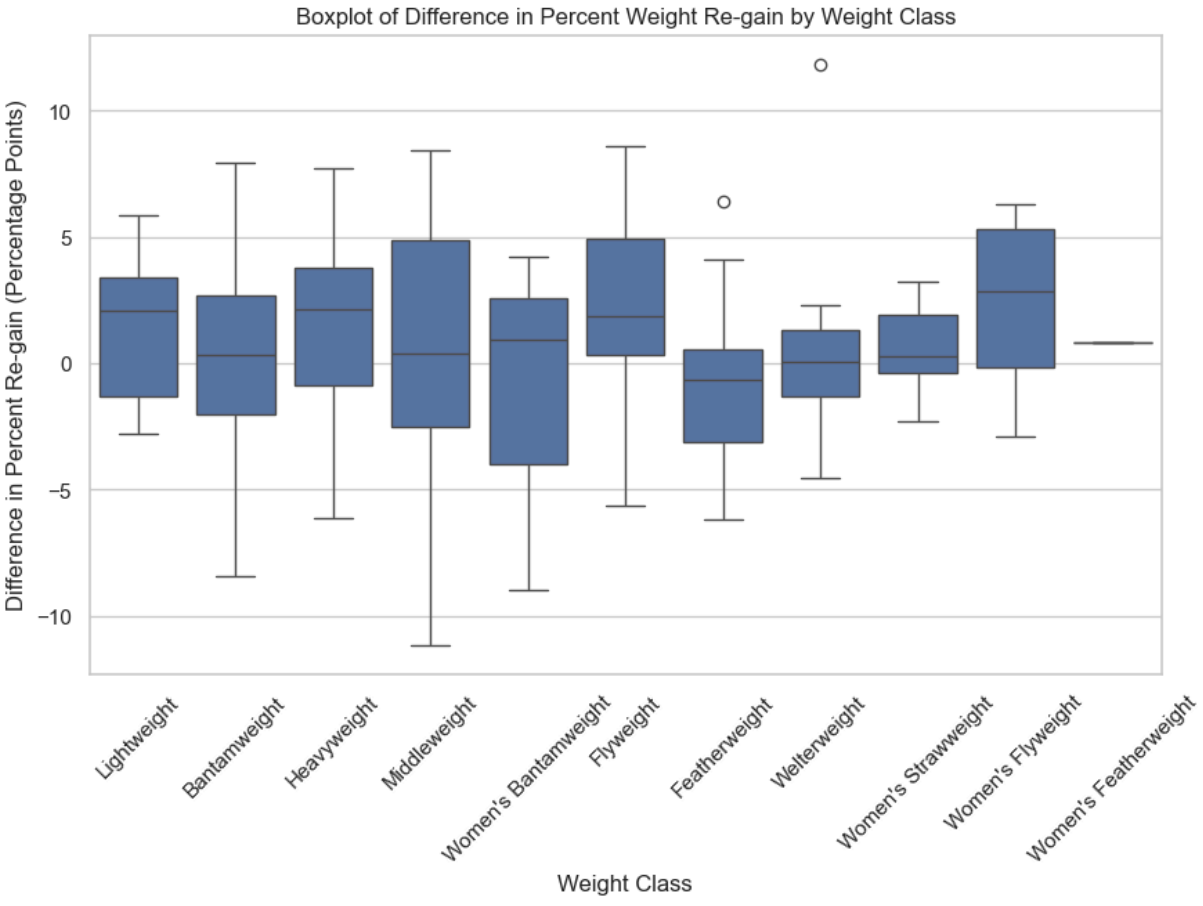
Exploratory Analysis

In [45]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as mticker
```

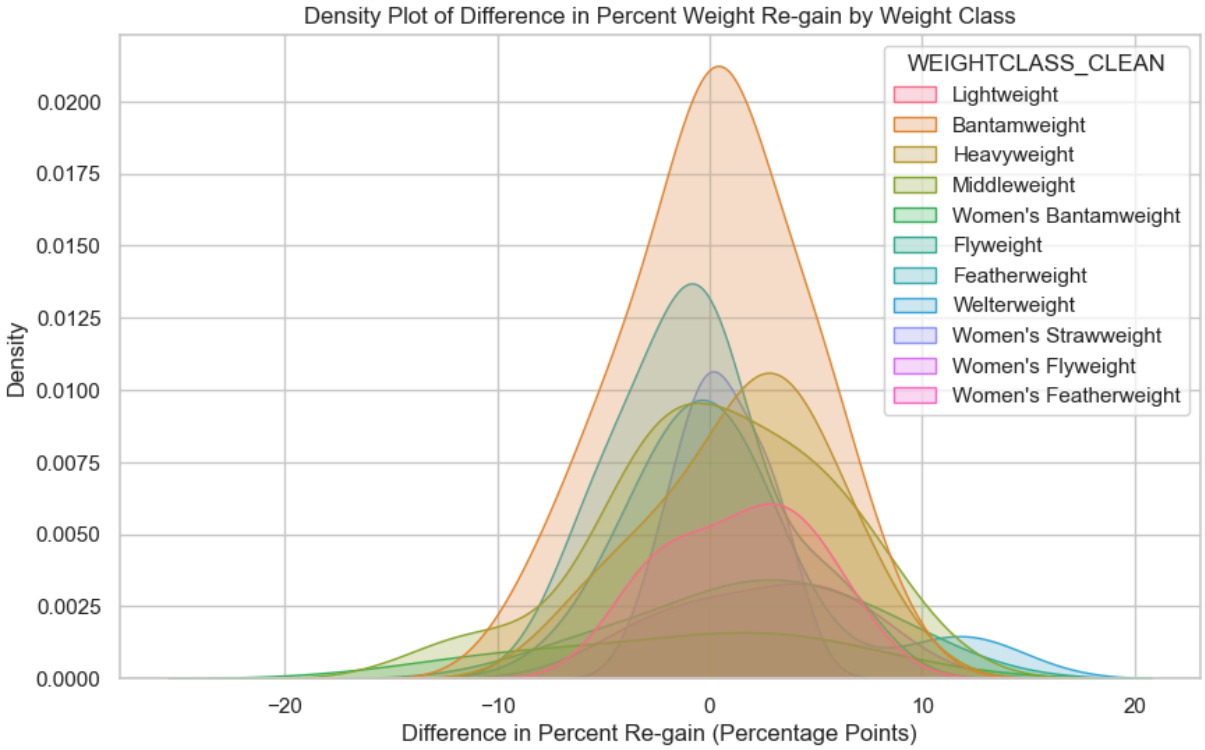
```python
import numpy as np


# ------------------------------
# Basic settings
# ------------------------------
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (10,6)

# ------------------------------
# 1. Histogram of Percent Re-gain (Overall)
# ------------------------------
plt.figure()
sns.histplot(df_no_duplicates_clean['PERCENT_REGAIN_DIFF'], bins=20, kde=Tru
plt.title("Histogram & Density of Difference in Percent Weight Re-gain (Over
plt.xlabel("Difference in Percent Re-gain (Percentage Points)")
plt.ylabel("Count")
plt.show()

# ------------------------------
# 2. Boxplot by Weight Class
# ------------------------------
plt.figure()
sns.boxplot(x='WEIGHTCLASS_CLEAN', y='PERCENT_REGAIN_DIFF', data=df_no_dupli
plt.title("Boxplot of Difference in Percent Weight Re-gain by Weight Class")
plt.xlabel("Weight Class")
plt.ylabel("Difference in Percent Re-gain (Percentage Points)")
plt.xticks(rotation=45)
plt.show()

# ------------------------------
# 3. Density curve by Weight Class
# ------------------------------
plt.figure()
sns.kdeplot(data=df_no_duplicates_clean, x='PERCENT_REGAIN_DIFF', hue='WEIGH
plt.title("Density Plot of Difference in Percent Weight Re-gain by Weight Cl
plt.xlabel("Difference in Percent Re-gain (Percentage Points)")
plt.ylabel("Density")
plt.show()

# ------------------------------
# 4. Beeswarm plot: Fighter vs Opponent Percent Regain
# ------------------------------
df_plot = df_no_duplicates_clean.copy()
df_plot['PERCENTAGE_REGAINED_ROUND'] = df_plot['PERCENT_REGAIN'].round(1)

sns.swarmplot(
    x='PERCENTAGE_REGAINED_ROUND',
    y='OPPONENT_PERCENTAGE_REGAINED',
    data=df_plot,
    size=4,
    alpha=0.6
)

plt.title("Beeswarm Plot: Fighter vs Opponent Percent Weight Re-gain")
plt.xlabel("Fighter Percent Re-gain")
```

```
plt.xlim(df_no_duplicates_clean['PERCENT_REGAIN'].min(), df_no_duplicates_cl
plt.xticks(ticks=range(0, 36, 5))  # ticks at 0, 5, 10, … 35%

plt.ylabel("Opponent Percent Re-gain")
plt.show()
```

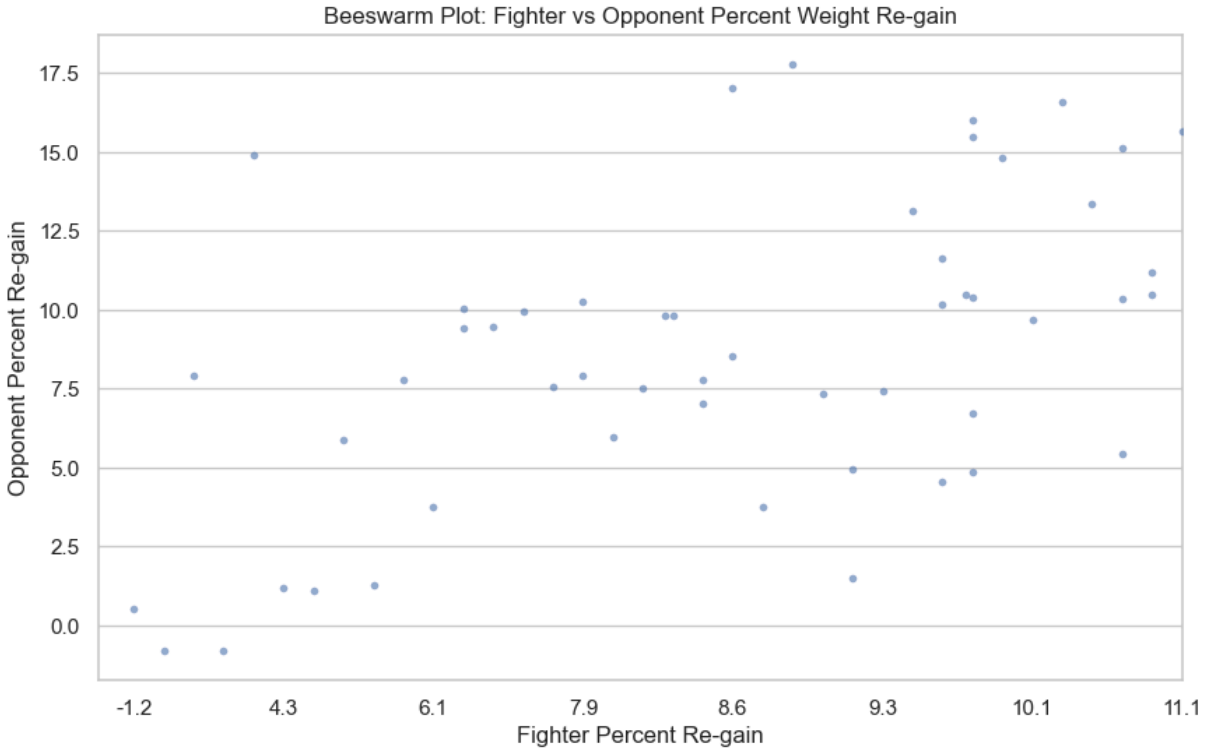




```
/var/folders/rc/yw7v7vhj4l3_vjznh0cwm8wr0000gn/T/ipykernel_68821/710180233.py:38: UserWarning: Dataset has 0 variance; skipping density estimate. Pass `warn_singular=False` to disable this warning.
  sns.kdeplot(data=df_no_duplicates_clean, x='PERCENT_REGAIN_DIFF', hue='WEIGHTCLASS_CLEAN', fill=True)
```




```
/Users/caseymoser/opt/anaconda3/envs/UFC_data/lib/python3.10/site-packages/seaborn/categorical.py:3399: UserWarning: 50.0% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
/Users/caseymoser/opt/anaconda3/envs/UFC_data/lib/python3.10/site-packages/seaborn/categorical.py:3399: UserWarning: 16.7% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
/Users/caseymoser/opt/anaconda3/envs/UFC_data/lib/python3.10/site-packages/seaborn/categorical.py:3399: UserWarning: 33.3% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



Choice of Regression

Given that the dependent variable is binary (win or loss), I have chosen to fit a logistic model to predict the log-odds of winning fights based on percentage weight regained.

In [48]:
```
#Logit Model

X = df_no_duplicates_clean[['PERCENT_REGAIN_DIFF']]


y = df_no_duplicates_clean['RESULT']

# Add a constant (intercept) to the independent variable
X = sm.add_constant(X)

# Fit the OLS model
model = sm.Logit(y, X)
result = model.fit()
print(result.summary())


math.exp(0.0485)
```

```
Optimization terminated successfully.
         Current function value: 0.687285
         Iterations 4
                           Logit Regression Results
==============================================================================
==
Dep. Variable:                 RESULT   No. Observations:
92
Model:                          Logit   Df Residuals:
90
Method:                           MLE   Df Model:
1
Date:                Mon, 01 Sep 2025   Pseudo R-squ.:                 0.0071
03
Time:                        20:54:29   Log-Likelihood:                -63.2
30
converged:                       True   LL-Null:                       -63.6
83
Covariance Type:            nonrobust   LLR p-value:                    0.34
15
==============================================================================
===========
                             coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
-----------
const                     0.0611      0.211      0.289      0.773      -0.353
0.475
PERCENT_REGAIN_DIFF       0.0485      0.051      0.944      0.345      -0.052
0.149
==============================================================================
===========
```

Out[48]: 1.049695371820325

In this first log-odds model, the coefficient on PERCENT_REGAIN is 0.0485, meaning every 1 percentage point fighter 1 increase in weight is linked to a 1.05 change in odds of winning. In other words, a 1% increase in weight regained following fight-weigh in is is linked to a 5% increase in odds of winning a fight holding all other variables constant.

This model is not statistically significant at alpha = 0.05, meaning that PERCENT_REGAIN_DIFF does not have a statistically signficant impact on winning at a 95% condifidence interval. Furthermore, given that the R^2 value is only 0.007, less than 1% of the variation in fight outcome is explained by percentage weight regain. In this way, this model has insufficient predictive power to be useful for fight prediction.

In [49]:
```python
# testing for causality on randomzied data in logistic model

from sklearn.model_selection import train_test_split

X = df_no_duplicates_clean[['PERCENT_REGAIN_DIFF']]

y = df_no_duplicates_clean['RESULT']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

### Logistic Regression
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)

print("Logistic Accuracy:", accuracy_score(y_test, y_pred_log))
print("Logistic AUC:", roc_auc_score(y_test, log_model.predict_proba(X_test)

### Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("\nRF Accuracy:", accuracy_score(y_test, y_pred_rf))
print("RF AUC:", roc_auc_score(y_test, rf_model.predict_proba(X_test)[:, 1])
```

```
Logistic Accuracy: 0.47368421052631576
Logistic AUC: 0.45238095238095244

RF Accuracy: 0.42105263157894735
RF AUC: 0.3333333333333333
```

In the logistic regression, the performance of the model further confirms that weight regain alone is insufficient to predict fight outcome. Since the AUC values are both less than 0.5, my models are worse than random guessing at predicting fight outcomes.

Regression with Squared Term

In my model, I want to investigate if doing big water cuts has a a signficant fall off past a point. As mentioned earlier, large water cuts can impact the health and performance of fighters, so the purpose of the squared term is to see if there is a point where doing too large of a water-cut negatively impacts the odds of a fighter winning.

In [50]:
```
df_no_duplicates_clean['PERCENT_REGAIN_SQ'] = df_no_duplicates_clean['PERCEN


X = df_no_duplicates_clean[['PERCENT_REGAIN_DIFF', 'PERCENT_REGAIN_SQ']]
X = sm.add_constant(X)

y = df_no_duplicates_clean['RESULT']


# Fit logistic regression
logit_model = sm.Logit(y, X)
results = logit_model.fit()

# Print summary
print(results.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.687241
         Iterations 4
                           Logit Regression Results
==============================================================================
==
Dep. Variable:                 RESULT   No. Observations:
92
Model:                          Logit   Df Residuals:
89
Method:                           MLE   Df Model:
2
Date:                Mon, 01 Sep 2025   Pseudo R-squ.:                 0.0071
66
Time:                        20:57:17   Log-Likelihood:                -63.2
26
converged:                       True   LL-Null:                       -63.6
83
Covariance Type:            nonrobust   LLR p-value:                    0.63
36
==============================================================================
===========
                            coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
-----------
const                     0.0737      0.254      0.290      0.772      -0.424
0.571
PERCENT_REGAIN_DIFF       0.0491      0.052      0.947      0.343      -0.052
0.151
PERCENT_REGAIN_SQ        -0.0008      0.008     -0.089      0.929      -0.017
0.016
==============================================================================
===========
```

In this third logistics model, the coefficients on PERCENT_REGAIN and PERCENT_REGAIN_SQ are both statsitically insignificant at a 95% and 90% confidence interval, implying that we fail to reject the null hypothesis that regaining weight has a downside at higher levels.

Analyzing Impact of Weight Regain on Odds of Winning by Weight Class

Certain weight classes may gain more benefit from weight regains. Based on my intutition, fighers in heavy weight do not have much to gain from drastic weight cuts, whereas the lower classes could see fighters who are a lot heavier do a drastic weight cut to have a signficant size advantadge relative to their competition. To explore this effect, I have done seperate regressions below by weight class.

In [53]:

```
# --- Get unique cleaned weight classes ---
weight_classes = df_no_duplicates_clean['WEIGHTCLASS_CLEAN'].dropna().unique

# ---  Run logistic regression for each weight class ---
results = {}

for wc in weight_classes:
    df_wc = df_no_duplicates_clean[df_no_duplicates_clean['WEIGHTCLASS_CLEAN

    # Drop missing data
    df_wc = df_wc.dropna(subset=['PERCENT_REGAIN_DIFF', 'RESULT'])

    # Check data sufficiency
    if len(df_wc) < 10 or df_wc['RESULT'].nunique() < 2:
        print(f"Skipping {wc} — insufficient data or outcome variation.")
        continue

    try:
        # Standardize the predictor
        df_wc['PERCENT_REGAIN_DIFF_STD'] = (
            df_wc['PERCENT_REGAIN_DIFF'] - df_wc['PERCENT_REGAIN_DIFF'].mean
        ) / df_wc['PERCENT_REGAIN_DIFF'].std()

        # Run logistic regression
        model = smf.logit("RESULT ~ PERCENT_REGAIN_DIFF_STD", data=df_wc).fi

        # Store result
        results[wc] = model.summary2().as_text()
        print(f"Model completed for {wc}")

    except Exception as e:
        print(f"⚠️ Error in {wc}: {e}")

# --- Print all regression summaries ---
print("\n" + "="*80)
print("LOGISTIC REGRESSION RESULTS BY WEIGHT CLASS")
print("="*80 + "\n")

for wc, summary in results.items():
    print(f"\n=== {wc.upper()} ===")
    print(summary)
    print("\n" + "-"*80)
```

```
Skipping Lightweight — insufficient data or outcome variation.
Model completed for Bantamweight
Model completed for Heavyweight
Model completed for Middleweight
Skipping Women's Bantamweight — insufficient data or outcome variation.
Skipping Flyweight — insufficient data or outcome variation.
Model completed for Featherweight
Skipping Welterweight — insufficient data or outcome variation.
Skipping Women's Strawweight — insufficient data or outcome variation.
Skipping Women's Flyweight — insufficient data or outcome variation.
Skipping Women's Featherweight — insufficient data or outcome variation.

================================================================================
====
LOGISTIC REGRESSION RESULTS BY WEIGHT CLASS
================================================================================
====


=== BANTAMWEIGHT ===
                           Results: Logit
=========================================================================
Model:                  Logit              Method:             MLE
Dependent Variable:     RESULT             Pseudo R-squared:   0.017
Date:                   2025-09-01 21:07   AIC:                32.3442
No. Observations:       22                 BIC:                34.5263
Df Model:               1                  Log-Likelihood:     -14.172
Df Residuals:           20                 LL-Null:            -14.421
Converged:              1.0000             LLR p-value:        0.48083
No. Iterations:         5.0000             Scale:              1.0000
-------------------------------------------------------------------------
                         Coef.  Std.Err.    z    P>|z|   [0.025 0.975]
-------------------------------------------------------------------------
Intercept                0.5736   0.4500  1.2746 0.2025 -0.3085 1.4556
PERCENT_REGAIN_DIFF_STD -0.3262   0.4708 -0.6929 0.4884 -1.2489 0.5965
=========================================================================


------------------------------------------------------------------------------
----

=== HEAVYWEIGHT ===
                           Results: Logit
=========================================================================
Model:                  Logit              Method:             MLE
Dependent Variable:     RESULT             Pseudo R-squared:   0.051
Date:                   2025-09-01 21:07   AIC:                16.2307
No. Observations:       11                 BIC:                17.0265
Df Model:               1                  Log-Likelihood:     -6.1153
Df Residuals:           9                  LL-Null:            -6.4455
Converged:              1.0000             LLR p-value:        0.41646
No. Iterations:         5.0000             Scale:              1.0000
-------------------------------------------------------------------------
                         Coef.  Std.Err.   z    P>|z|   [0.025 0.975]
-------------------------------------------------------------------------
Intercept                1.0460   0.7160 1.4610 0.1440 -0.3573 2.4492
```

```
PERCENT_REGAIN_DIFF_STD 0.5813   0.7330 0.7931 0.4277 -0.8553 2.0179
=================================================================


-----------------------------------------------------------------------------
----

=== MIDDLEWEIGHT ===
                           Results: Logit
=================================================================
Model:                 Logit             Method:            MLE
Dependent Variable:    RESULT            Pseudo R-squared:  0.367
Date:                  2025-09-01 21:07  AIC:               15.3540
No. Observations:      13                BIC:               16.4839
Df Model:              1                 Log-Likelihood:    -5.6770
Df Residuals:          11                LL-Null:           -8.9724
Converged:             1.0000            LLR p-value:       0.010251
No. Iterations:        7.0000            Scale:             1.0000
-----------------------------------------------------------------
                        Coef.  Std.Err.    z    P>|z|   [0.025 0.975]
-----------------------------------------------------------------
Intercept               0.2094   0.7431 0.2819 0.7780 -1.2469 1.6658
PERCENT_REGAIN_DIFF_STD 2.2971   1.2724 1.8054 0.0710 -0.1967 4.7909
=================================================================


-----------------------------------------------------------------------------
----

=== FEATHERWEIGHT ===
                           Results: Logit
=================================================================
Model:                 Logit             Method:            MLE
Dependent Variable:    RESULT            Pseudo R-squared:  0.058
Date:                  2025-09-01 21:07  AIC:               16.7167
No. Observations:      12                BIC:               17.6865
Df Model:              1                 Log-Likelihood:    -6.3583
Df Residuals:          10                LL-Null:           -6.7480
Converged:             1.0000            LLR p-value:       0.37733
No. Iterations:        6.0000            Scale:             1.0000
-----------------------------------------------------------------
                        Coef.  Std.Err.    z    P>|z|   [0.025 0.975]
-----------------------------------------------------------------
Intercept               -1.1991   0.7269 -1.6495 0.0990 -2.6238 0.2257
PERCENT_REGAIN_DIFF_STD -0.6749   0.8187 -0.8244 0.4097 -2.2795 0.9297
=================================================================


-----------------------------------------------------------------------------
----
```

In [58]:
```
print(math.exp(2.2971))

print(100*(math.exp(2.2971)-1))
```

```
9.945299226618967
894.5299226618968
```

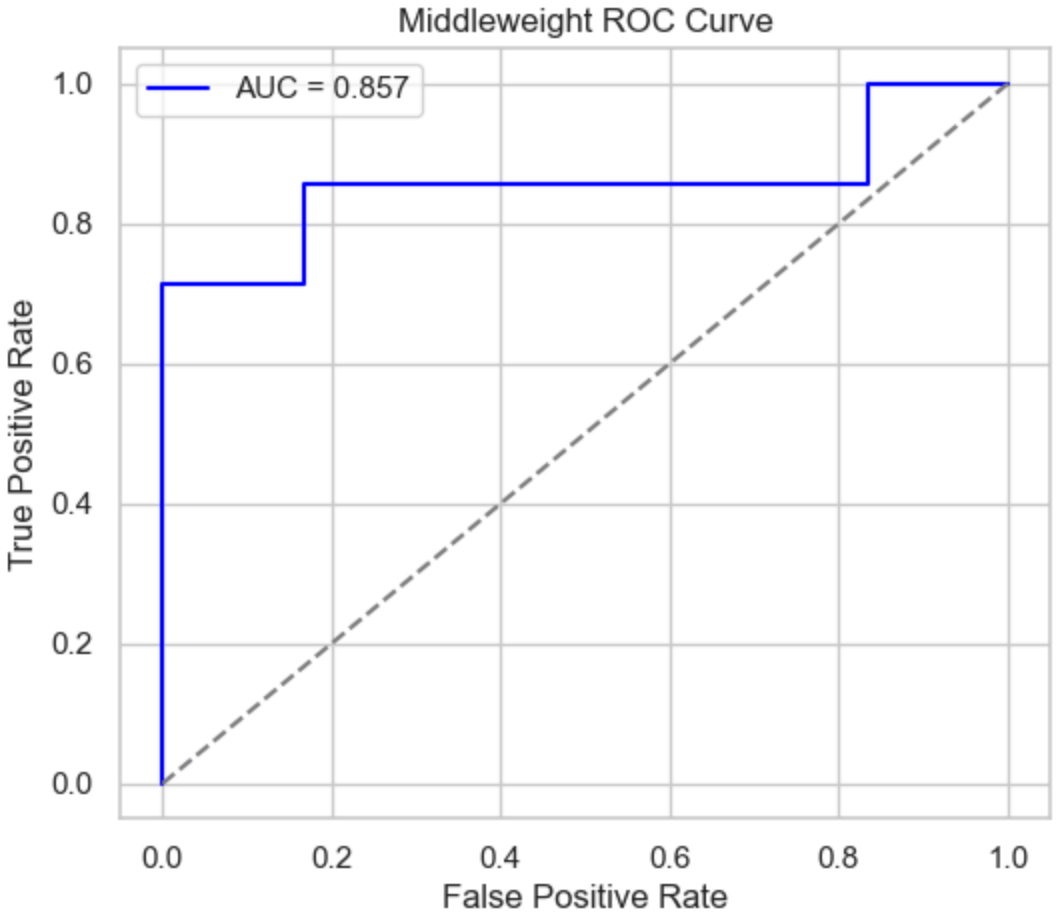
For fighters in the middleweight division, every 1 percentage point more weight a fighter has regained compared to their opponent is linked to a 2.2971 change in their odds of winning. In other words, each 1 percentage point more weight a fighter has regained relative to their opponent is linked to a 894.53% increase in odds of winning a fight holding all other variables constant. This result is statistically signficant at a 90% confidence level. Intutitively, this increase in odds makes sense because of the middleweight division's relative position compared to other weight classes. Many of the competitive middleweights, such as Alex Pereira who is 6"4 and walks around at 220+ pounds, are big enough to fight in the light-heavyweight division. These fighters that make the cut to middleweight are signficantly bigger than their competitors at middleweight. On the otherhand, there are also fighters in the middleweight division that are too heavy to fight in welterweight so they end up moving to middleweight to be more competitive, such as Robert Whittiker or Sean Strickland, both of whom were former champions at middleweight but were uncompetitive at welterweight. Since middleweight exists in an awkward position between welterweight and light-heavyweight, it could be the case that the middle weights that have bigger frames (in other words the ones that can cut more weight) have a statistically significant advantadge over the middleweights that have smaller frames and are unable to cut as much weight.

In [64]:
```python
def likelihood_ratio_test(full_model):
    llf_full = full_model.llf
    llf_null = full_model.llnull
    df_full = full_model.df_model
    df_null = 0  # Null model has only intercept

    lr_stat = 2 * (llf_full - llf_null)
    p_value = chi2.sf(lr_stat, df=df_full)

    return {"LR statistic": lr_stat, "df": df_full, "p-value": p_value}

def plot_roc_auc(model, data, title="ROC Curve"):
    data = data.copy()
    y_true = data['RESULT']
    y_scores = model.predict()

    fpr, tpr, thresholds = roc_curve(y_true, y_scores)
    auc = roc_auc_score(y_true, y_scores)

    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}", color='blue')
    plt.plot([0, 1], [0, 1], '--', color='gray')
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
    return auc
```

In [65]:
```
df_middleweight = df_no_duplicates_clean[df_no_duplicates_clean['WEIGHTCLASS

# Standardize predictor
df_middleweight['PERCENT_REGAIN_DIFF_STD'] = (
    df_middleweight['PERCENT_REGAIN_DIFF'] - df_middleweight['PERCENT_REGAIN
) / df_middleweight['PERCENT_REGAIN_DIFF'].std()

# Fit model
model_middle = smf.logit("RESULT ~ PERCENT_REGAIN_DIFF_STD", data=df_middlew

# Likelihood Ratio Test
print("Likelihood Ratio Test:")
print(likelihood_ratio_test(model_middle))


# ROC Curve & AUC
print("\nROC Curve and AUC:")
auc = plot_roc_auc(model_middle, df_middleweight, title="Middleweight ROC Cu
print(f"AUC: {auc:.3f}")
```

```
Likelihood Ratio Test:
{'LR statistic': np.float64(6.59083104870467), 'df': 1.0, 'p-value': np.floa
t64(0.010250530960438377)}

ROC Curve and AUC:
```



```
AUC: 0.857
```

The AUC value for the model for the Middleweight division is 0.857. This value suggests that the model is better at predicting which fighter wins than random guessing. Since the AUC value is close to 1, my model is a relatively strong predictor of fight outcomes in the middleweight division.

Additionally, the likelihood ratio statistic is statistically signficant at a 95% confidence interval, meaning that middleweight fighters that regain a greater percentage of weight compared to their opponents between weigh-in and fight night are signficantly increasing their odds of winning the fight with all other variables being held constant.

Limitation to Analysis of Weight Regain Models and Room to Expand

The biggest limitation to this analysis is the data set. Not all regions require fighters to publicize their fight night weight, meaning a lot of weight regains are not capture in my analysis. Additionally, my current fighter weight data is from 6 years ago, meaning that the last 6 years of fights have not been captured by the data. Although my results are statistically signficant, my data set for midddlweight only includes 13 fights total, which is highly limited. In this way, the extreme effect of having regained a greater percentage of weight compared to an opponent in the middleweight divsion is potentially over-exaggerated because of the small sample size. Having more post fight weigh in data could let us more accurately assess if there is a limitation to the benefit of cutting and regaining weight.

In [ ]: